

# Pandas & scikit-learn Patterns You'll See This Course — Session 6

Machine Learning in Geoscience — pre-session reference · read before Session 6

Sessions 1-5 built and evaluated models. Session 6 asks *what the model learned* and *where it fails* — so the new patterns are about pulling numbers out of an already-fitted model (importances) and about summarizing errors by category (regime breakdowns). Nothing here trains a new kind of model; it all runs on models you already know how to build.

## 1. Feature importance, two ways

`rf.feature_importances_`

After fitting, a tree-based model exposes this array: one number per feature, measuring how much it reduced error across all trees. It's computed from the **training** data, so it can overrate a feature the model merely memorized.

```
importance = pd.Series(rf.feature_importances_,
                      index=D['features'])
importance.sort_values(ascending=False).head(20)
```

`permutation_importance(model, X_val, y_val, n_repeats=5, scoring=...)`

A different question: shuffle one column in **held-out** data and see how much worse the error gets. Returns an object, not a plain array — pull the numbers out as `.importances_mean` and `.importances_std`.

```
from sklearn.inspection import permutation_importance
perm = permutation_importance(rf, X_val, y_val, n_repeats=5,
                             scoring='neg_mean_absolute_error')
perm.importances_mean    # one number per feature
perm.importances_std     # how noisy that estimate is
```

## 2. Named aggregation in groupby

`df.groupby('hour').agg(MAE=('abs_error', 'mean'), n=('abs_error', 'size'))`

A more explicit form of `.groupby().mean()`: each keyword becomes an output column name, paired with `(source_column, function)`. Lets you compute several different summaries from different columns in one call — here, error broken down by hour of day, month, or wind-speed bucket.

## Before Session 6 — a 10-minute self-check

- Fit any scikit-learn tree model on toy data and print `model.feature_importances_`.
- Try a `groupby(...).agg(x=('col', 'mean'), n=('col', 'size'))` call on a toy DataFrame and check that the output column names match what you asked for.
- If something above is still unclear, that's fine — flag it at the start of the session instead of getting stuck mid-lecture.

Machine Learning in Geoscience — Session 6 pre-read. This sheet covers recognition only; the course notebook is where these patterns get used for real, with full explanations in context.

## 3. Defensive code around optional / shifting APIs

`try: a() except Exception: b()`

Used here because SHAP's API changed between versions — one line works on older installs, a different line on newer ones. The pattern "try the common case, fall back on failure" shows up anywhere a library isn't guaranteed to behave identically across versions.

`arr = np.asarray(x); if arr.ndim == 3: arr = arr[:, :, 0]`

Checking `.ndim` (how many axes an array has) before indexing it, because some versions of a library return a 2-D array and others a 3-D one for the same call. Reading this: "if there's an extra third axis, keep only its first slice."

## 4. Freeing memory explicitly

`import gc / gc.collect()`

This session fits several large models back to back (one per horizon, for the regime analysis) inside one notebook run. `del model` (seen since Session 4) drops the reference; `gc.collect()` asks Python to actually reclaim that memory right away, rather than waiting.

## 5. Reading a residual convention

`residual = predicted - observed`

A sign convention, not a new function — but easy to misread if you assume the opposite. Here, positive means the model **over**-forecast. Every "bias" and "mean residual" number in this session follows this convention.